templates in cpp

Templates in C++ (Simple Explanation)

Templates in C++ allow you to write **generic code** that works with **different data types** without rewriting the same logic again and again.

In simple words:
Template = A blueprint for functions or classes that can work with any data type.

Why we use Templates

Without templates, you would have to write the same function for each data type.

Without Template

```cpp
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}
```

You must write the same logic multiple times.

With Template

```cpp
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

Now the same function works for:

- int
- float
- double
- etc.

Example Program

```cpp
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b) {
    return a + b;
}

int main() {
    cout << add(5, 3) << endl;        // int
    cout << add(4.5, 2.3) << endl;    // double
    return 0;
}
```

Output

```
8
6.8
```

How Template Works (Step by Step)

1. `template <typename T>`
   - This tells the compiler:
   - "I am creating a generic function using a type named **T**."
2. `T add(T a, T b)`
   - Function will take two values of type **T**.
   - It returns type **T**.
3. When you call:

   ```cpp
   add(5, 3);
   ```

   Compiler automatically replaces:

   ```ini
   T = int
   ```

4. When you call:

   ```cpp
   add(4.5, 2.3);
   ```

   Compiler replaces:

   ```ini
   T = double
   ```

Types of Templates

1. Function Template

Used for functions.

```cpp
template <class T>
T multiply(T a, T b) {
    return a * b;
}
```

---

2. Class Template

Used for classes.

```cpp
#include <iostream>
using namespace std;

template <class T>
class Box {
public:
    T value;

    void setValue(T v) {
        value = v;
    }

    void show() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Box<int> intBox;
    intBox.setValue(10);
    intBox.show();

    Box<string> strBox;
    strBox.setValue("Hello");
    strBox.show();

    return 0;
}
```

`typename` **vs** `class`

Both are the same in templates.

```cpp
template <typename T>
```

or

```cpp
template <class T>
```

No difference in function templates.

Multiple Template Parameters

```cpp
template <typename T, typename U>
void show(T a, U b) {
    cout << a << " " << b;
}
```

Usage:

```cpp
show(5, "Hello");
```

Key Advantages

- Reusable code
- Less repetition
- Type safety
- More flexible programs

Real-life Example

Template is like a **cookie cutter**:

- Same shape
- But you can use different dough (chocolate, vanilla, etc.)

If you want, I can:

- Explain templates with **step-by-step memory diagrams**
- Give **exam-style questions and answers**
- Compare **templates vs macros vs function overloading**
- Show **real projects using templates**